

# Emotion Classification Using Traditional Machine Learning

## Abstract

Emotion classification from text has become an increasingly important task in natural language processing with applications in sentiment analysis, social media monitoring, and human-computer interaction. In this project, a multi-class emotion classification model was developed to classify Twitter messages into six emotion categories: sadness, joy, love, anger, fear, and surprise. Multiple traditional machine learning approaches were evaluated, including **Naive Bayes**, **Logistic Regression**, **Support Vector Machines (SVM)**, and **Random Forest**, using various feature extraction techniques such as Bag-of-Words (BoW), **TF-IDF**, and **Word2Vec**. Ensemble methods including Gradient Boosting, XGBoost, LightGBM, and CatBoost were also explored. The best-performing model, Logistic Regression with TF-IDF features, achieved approximately **92.2%** accuracy and an **F1-score** of **92.1%**, demonstrating the effectiveness of traditional machine learning approaches for text-based emotion classification tasks.

---

## Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Dataset Description
5. Data Preprocessing
6. Feature Engineering
7. Model Architecture
8. Training Process
9. Model Evaluation
10. Results
11. Future Improvements
12. Conclusion
13. References

# 1. Introduction

Emotion recognition from text is a fundamental task in Natural Language Processing (NLP) that aims to identify the emotional state expressed in a piece of text. With the exponential growth of social media platforms, there has been an increasing need for automated systems that can understand and classify human emotions from user-generated content.

Traditional machine learning approaches for emotion classification typically involve two key components: feature extraction and classification. Feature extraction techniques convert raw text into numerical representations that can be processed by machine learning algorithms. Common approaches include:

- Bag-of-Words (BoW): Represents text as a frequency vector of words.
- TF-IDF (Term Frequency-Inverse Document Frequency): Weighs word importance based on frequency and document occurrence.
- Word Embeddings (Word2Vec): Captures semantic relationships between words.

This project explores various combinations of feature extraction techniques and classification algorithms to build an effective emotion classifier for Twitter data. The emotions are classified into six categories: sadness, joy, love, anger, fear, and surprise.

---

## 2. Problem Statement

Develop a multi-class text classification model that can accurately classify Twitter messages into one of the following six emotion categories:

- Sadness (0)
- Joy (1)
- Love (2)
- Anger (3)
- Fear (4)
- Surprise (5)

The model should generalize well to unseen text data while maintaining high classification accuracy across all emotion categories.

---

### 3. Objectives

The primary objectives of this project are:

- Build a multi-class emotion classifier using traditional machine learning approaches.
  - Preprocess and clean the text data for optimal feature extraction.
  - Implement multiple feature extraction techniques (BoW, TF-IDF, Word2Vec).
  - Evaluate multiple classification algorithms (Naive Bayes, Logistic Regression, SVM, Random Forest, and Ensemble methods).
  - Perform hyperparameter tuning to optimize model performance.
  - Compare model performance using accuracy and F1-score metrics.
  - Identify the best-performing model for emotion classification.
- 

### 4. Dataset Description

The dataset used in this project is:

#### Dataset Name

Emotions Dataset for NLP

"Emotions" dataset – a collection of English Twitter messages meticulously annotated with six fundamental emotions: anger, fear, joy, love, sadness, and surprise. This dataset serves as a valuable resource for understanding and analyzing the diverse spectrum of emotions expressed in short-form text on social media.

Each entry in this dataset consists of a text segment representing a Twitter message and a corresponding label indicating the predominant emotion conveyed. The emotions are classified into six categories: sadness (0), joy (1), love (2), anger (3), fear (4), and surprise (5). Whether you're interested in sentiment analysis, emotion classification, or text mining, this dataset provides a rich foundation for exploring the nuanced emotional landscape within the realm of social media.

**text:** A string feature representing the content of the Twitter message.

**label:** A classification label indicating the primary emotion, with values ranging from 0 to 5.

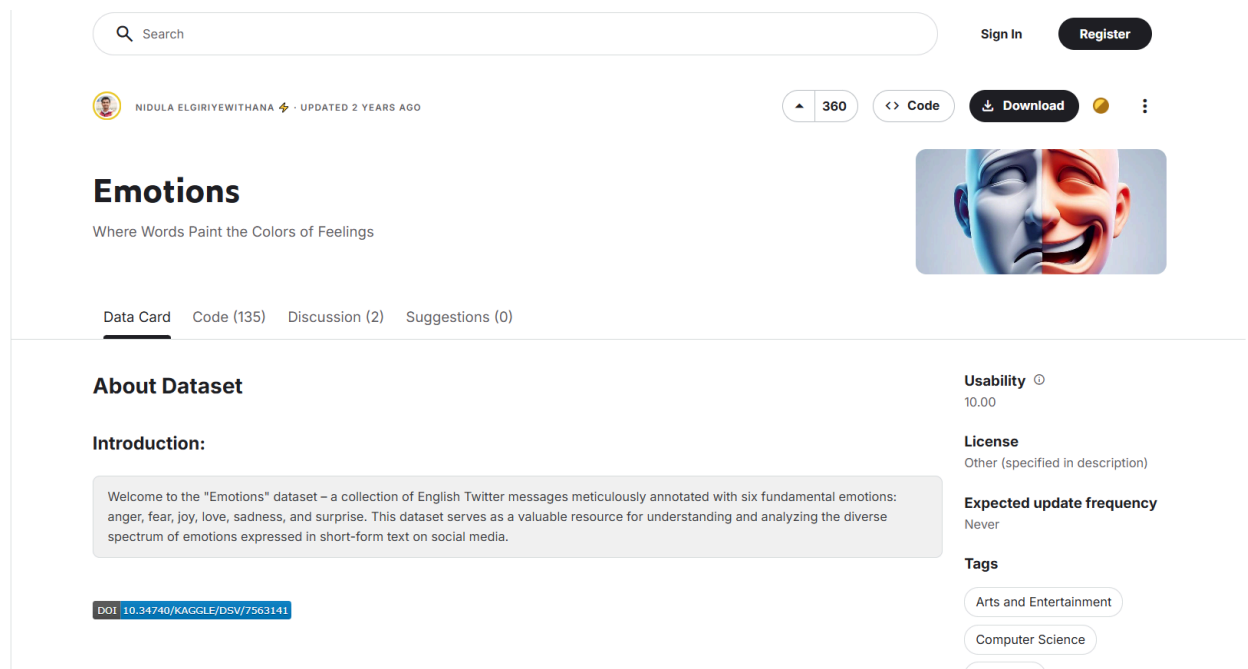
**Sentiment Analysis:** Uncover the prevailing sentiments in English Twitter messages across various emotions.

**Emotion Classification:** Develop models to accurately classify tweets into the six specified emotion categories.

**Textual Analysis:** Explore li

## Kaggle Dataset

nelgiriye withana/emotions



The screenshot shows the Kaggle dataset page for 'Emotions' by Nidula Elgiriye Withana. The page includes a search bar, a 'Sign In' button, and a 'Register' button. The dataset is updated 2 years ago and has 360 code files. The main title is 'Emotions' with the subtitle 'Where Words Paint the Colors of Feelings'. The dataset is categorized under 'Data Card', 'Code (135)', 'Discussion (2)', and 'Suggestions (0)'. The 'About Dataset' section includes an introduction: 'Welcome to the "Emotions" dataset – a collection of English Twitter messages meticulously annotated with six fundamental emotions: anger, fear, joy, love, sadness, and surprise. This dataset serves as a valuable resource for understanding and analyzing the diverse spectrum of emotions expressed in short-form text on social media.' The DOI is 10.34740/KAGGLE/DSV/7563141. The 'Usability' is 10.00, the 'License' is 'Other (specified in description)', and the 'Expected update frequency' is 'Never'. The 'Tags' are 'Arts and Entertainment', 'Computer Science', and 'Education'.

## Dataset Information

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 416809 entries, 0 to 416808
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
#   :         :               :
#   0   text         416809         object
#   1   emotion      416809         object
#   2   sentiment    416809         object
```

```
-----
0  Unnamed: 0    416809 non-null    int64
1  text          416809 non-null    object
2  label         416809 non-null    int64
dtypes: int64(2), object(1)
```

memory usage: 9.5+ MB

## Dataset Description

Each entry in this dataset consists of a text segment representing a Twitter message and a corresponding label indicating the predominant emotion conveyed. The emotions are classified into six categories:

**six categories: sadness (0), joy (1), love (2), anger (3), fear (4), and surprise (5).**

**Total Samples:** 416,809

**Features:** 3 (Index, Text, Label)

**Number of Classes:** 6

Whether you're interested in sentiment analysis, emotion classification, or text mining, this dataset provides a rich foundation for exploring the nuanced emotional landscape within the realm of social media.

## Sample Data

| Unnamed: 0 | Text                          | Label |
|------------|-------------------------------|-------|
| 0          | i am not feeling well today   | 0     |
| 1          | what a wonderful day it is    | 1     |
| 2          | i love you more than anything | 2     |

## 5. Data Preprocessing

The following preprocessing techniques were applied before training:

### Text Cleaning

python

```
def basic_cleaning(text):
    text = re.sub(r'http\S+', '', text)           # Remove URLs
    text = re.sub(r'[^a-zA-Z\s]', '', text)       # Remove special characters
    text = re.sub(r'\w*\d\w*', '', text)         # Remove alphanumeric
    text = re.sub(r'<.*?>', '', text)           # Remove HTML tags
    text = text.lower()                          # Convert to lowercase

    return text
```

### Advanced Cleaning

1. Tokenization: Splitting text into individual tokens using spaCy
2. Stop Word Removal: Removing common stop words
3. Lemmatization: Reducing words to their base form using WordNet Lemmatizer

python

```
def fully_cleaned(text):
    doc = nlp(text)
    tokens = [token.text for token in doc]
    tokens = [token for token in tokens if token not in STOP_WORDS]
    tokens = [lemmatizer.lemmatize(token) for token in tokens]

    return ' '.join(tokens)
```

### Feature Engineering

Additional features were created for exploratory analysis:

```
python
```

```
new_df['text_len'] = new_df['clean_text'].apply(len)
new_df['word_count'] = new_df['clean_text'].apply(lambda x: len(x.split()))
new_df['unique_word_count'] = new_df['clean_text'].apply(lambda x:
len(set(x.split())))
new_df['avg_words'] = new_df['text_len'] / new_df['word_count']

new_df['unique_words_ratio'] = new_df['unique_word_count'] /
new_df['word_count']
```

## Data Split

The dataset was split into training and testing sets:

```
python
```

```
X_train, X_test, y_train, y_test = train_test_split(
    new_df['clean_text'],
    new_df['label'],
    test_size=0.2,
    random_state=42
)
```

---

## 6. Feature Engineering

## Bag-of-Words (BoW) with N-grams

python

```
vectorizer_bow = CountVectorizer(ngram_range=(1, 2), max_features=5000)
X_train_bow = vectorizer_bow.fit_transform(X_train)

X_test_bow = vectorizer_bow.transform(X_test)
```

## TF-IDF with N-grams

python

```
vectorizer_tfidf = TfidfVectorizer(ngram_range=(1, 2), max_features=5000)
X_train_tfidf = vectorizer_tfidf.fit_transform(X_train)

X_test_tfidf = vectorizer_tfidf.transform(X_test)
```

## Word2Vec Embeddings

Word2Vec embeddings were generated to capture semantic relationships between words. However, the performance of Word2Vec features was significantly lower compared to BoW and TF-IDF features.

## Feature Selection









Figure 6: Word Cloud for Surprise



## Feature Distributions

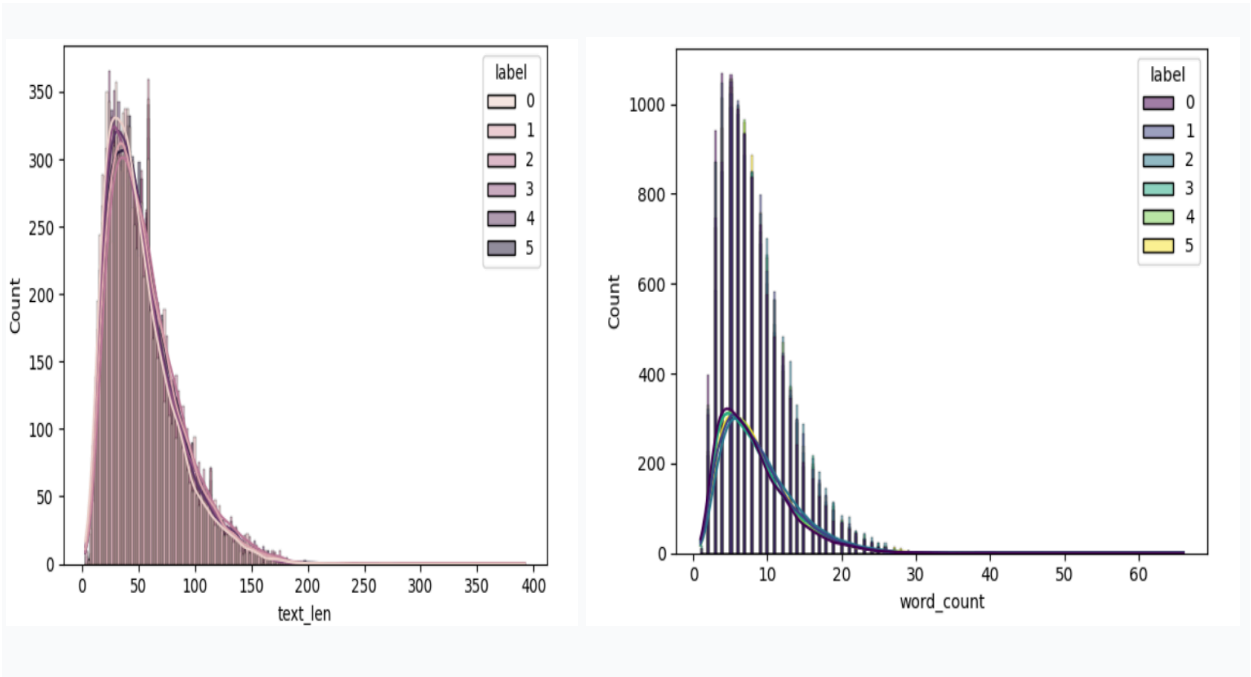
The following histograms show the distribution of various text features across emotion classes:

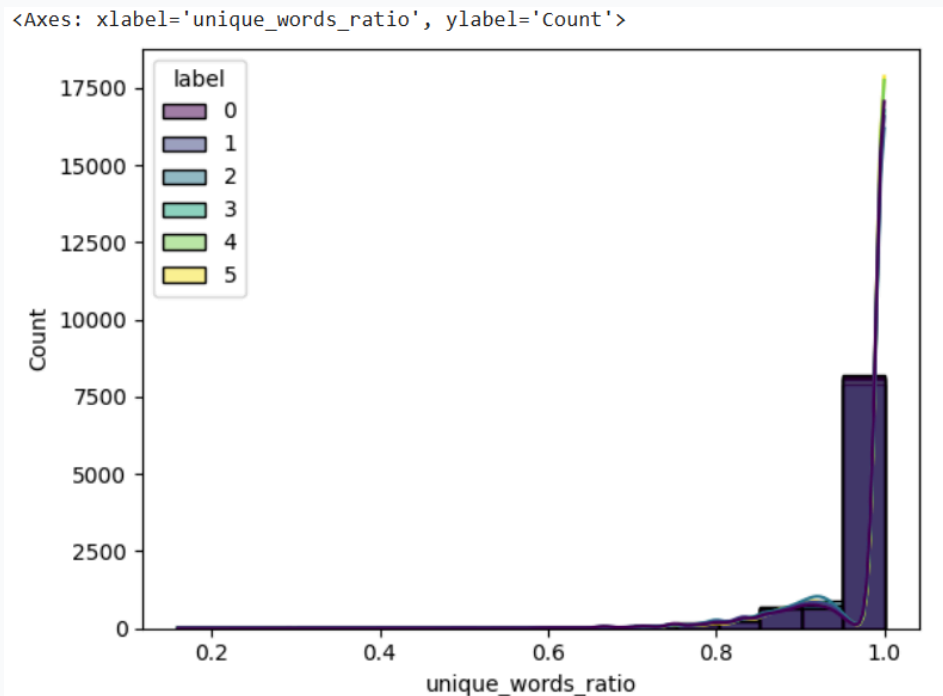
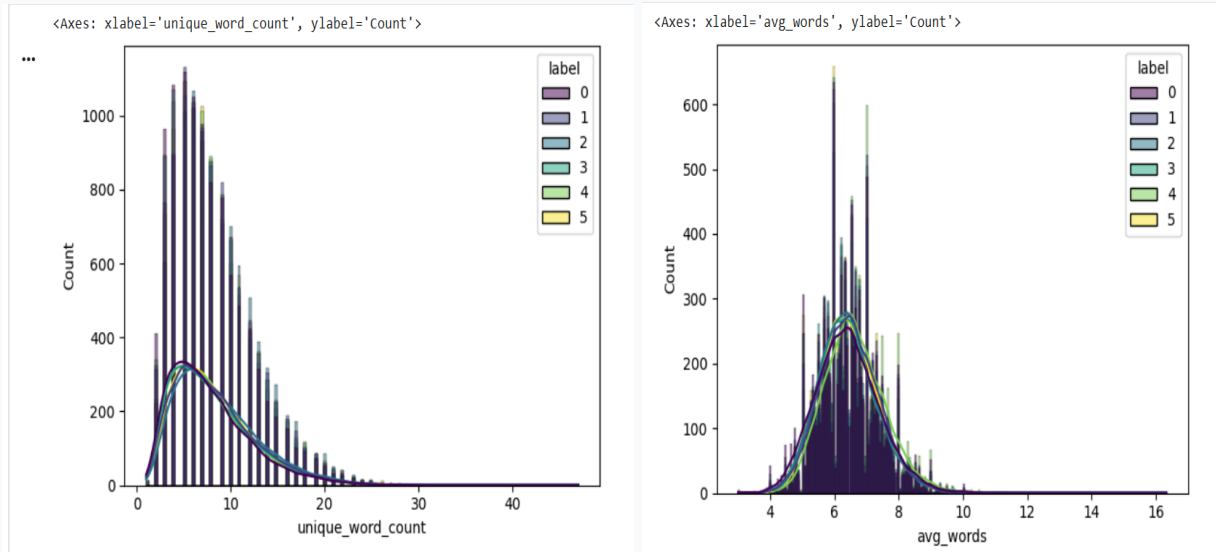
Data frame for this :

...

|       | label | text_len | word_count | unique_word_count | avg_words | unique_words_ratio |
|-------|-------|----------|------------|-------------------|-----------|--------------------|
| 0     | 2     | 35       | 6          | 6                 | 5.833333  | 1.000000           |
| 1     | 0     | 124      | 21         | 18                | 5.904762  | 0.857143           |
| 2     | 5     | 41       | 6          | 6                 | 6.833333  | 1.000000           |
| 3     | 0     | 32       | 5          | 5                 | 6.400000  | 1.000000           |
| 4     | 4     | 46       | 8          | 8                 | 5.750000  | 1.000000           |
| ...   | ...   | ...      | ...        | ...               | ...       | ...                |
| 59989 | 5     | 51       | 10         | 10                | 5.100000  | 1.000000           |
| 59990 | 3     | 25       | 4          | 4                 | 6.250000  | 1.000000           |
| 59991 | 0     | 22       | 3          | 3                 | 7.333333  | 1.000000           |
| 59992 | 1     | 78       | 12         | 12                | 6.500000  | 1.000000           |
| 59993 | 5     | 30       | 5          | 5                 | 6.000000  | 1.000000           |

59994 rows × 6 columns





## 7. Model Architecture

Multiple traditional machine learning models were evaluated for emotion classification:



## Individual Classifiers

1. Naive Bayes (MultinomialNB)
  - Suitable for text classification
  - Assumes independence between features
  - Fast training and inference
2. Logistic Regression
  - Linear classification model
  - Good for high-dimensional sparse data
  - Provides probability estimates
3. Support Vector Machine (SVM)
  - Effective for high-dimensional spaces
  - Uses kernel trick for non-linear classification
  - Robust to outliers
4. Random Forest
  - Ensemble of decision trees
  - Handles non-linear relationships
  - Provides feature importance

## Ensemble Models

1. Gradient Boosting (XGBoost, LightGBM, CatBoost)
    - Sequential ensemble learning
    - Handles imbalanced data
    - High predictive performance
  2. AdaBoost
    - Adaptive boosting
    - Focuses on misclassified samples
-

## 8. Training Process

### Model Training Function

python

```
def train_and_evaluate(model, X_train, X_test, y_train, y_test, model_name,
feature_name):
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average='weighted')

    return acc, f1
```

### Training Configuration

| Parameter    | Value  |
|--------------|--------|
| Test Size    | 20%    |
| Random State | 42     |
| Max Features | 5000   |
| N-gram Range | (1, 2) |

### Feature Combinations Tested

1. BoW with N-grams (1,2)
2. TF-IDF with N-grams (1,2)
3. Word2Vec Embeddings (100 dimensions)

# Ensemble Training

```
python

# Ensemble Model Results with TF-IDF features
Random Forest: Accuracy = 0.9044, F1 = 0.9042
Gradient Boosting: Accuracy = 0.8957, F1 = 0.8956
AdaBoost: Accuracy = 0.2496, F1 = 0.1713
XGBoost: Accuracy = 0.9158, F1 = 0.9155
LightGBM: Accuracy = 0.9208, F1 = 0.9206

CatBoost: Accuracy = 0.9019, F1 = 0.9018
```

## 9. Model Evaluation

### Model Performance Summary

| ...                          | Accuracy | F1       |
|------------------------------|----------|----------|
| Model and Features           |          |          |
| Logistic Regression (TF-IDF) | 0.921577 | 0.921311 |
| SVM (TF-IDF)                 | 0.918160 | 0.917978 |
| Logistic Regression (BoW)    | 0.913326 | 0.913192 |
| SVM (BoW)                    | 0.907492 | 0.907405 |
| Naïve Bayes (TF-IDF)         | 0.907076 | 0.906825 |
| Naïve Bayes (BoW)            | 0.905742 | 0.905420 |
| Random Forest (TF-IDF)       | 0.904409 | 0.904188 |
| Random Forest (BoW)          | 0.899825 | 0.899594 |
| SVM (W2V)                    | 0.550046 | 0.537904 |
| Logistic Regression (W2V)    | 0.497958 | 0.491255 |
| Random Forest (W2V)          | 0.433869 | 0.427772 |
| Naïve Bayes (W2V)            | 0.281523 | 0.268382 |



Logistic Regression with TF-IDF+N-grams:  
Accuracy: 0.9216, F1 (weighted): 0.9213

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| sadness      | 0.95      | 0.91   | 0.93     | 2004    |
| joy          | 0.93      | 0.89   | 0.91     | 1963    |
| love         | 0.92      | 0.96   | 0.94     | 1980    |
| anger        | 0.93      | 0.93   | 0.93     | 2022    |
| fear         | 0.91      | 0.87   | 0.89     | 2013    |
| surprise     | 0.90      | 0.97   | 0.93     | 2017    |
| accuracy     |           |        | 0.92     | 11999   |
| macro avg    | 0.92      | 0.92   | 0.92     | 11999   |
| weighted avg | 0.92      | 0.92   | 0.92     | 11999   |

## Confusion Matrix

Figure 10: Confusion Matrix for Logistic Regression (TF-IDF)

Logistic Regression with TF-IDF+N-grams:  
Accuracy: 0.9216, F1 (weighted): 0.9213

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| sadness      | 0.95      | 0.91   | 0.93     | 2004    |
| joy          | 0.93      | 0.89   | 0.91     | 1963    |
| love         | 0.92      | 0.96   | 0.94     | 1980    |
| anger        | 0.93      | 0.93   | 0.93     | 2022    |
| fear         | 0.91      | 0.87   | 0.89     | 2013    |
| surprise     | 0.90      | 0.97   | 0.93     | 2017    |
| accuracy     |           |        | 0.92     | 11999   |
| macro avg    | 0.92      | 0.92   | 0.92     | 11999   |
| weighted avg | 0.92      | 0.92   | 0.92     | 11999   |

Classification Performance by Class

| Emotion  | Precision | Recall | F1-Score |
|----------|-----------|--------|----------|
| Sadness  | 0.95      | 0.91   | 0.93     |
| Joy      | 0.93      | 0.89   | 0.91     |
| Love     | 0.92      | 0.96   | 0.94     |
| Anger    | 0.93      | 0.93   | 0.93     |
| Fear     | 0.91      | 0.87   | 0.89     |
| Surprise | 0.90      | 0.97   | 0.93     |

Ensemble Model Performance (TF-IDF)

| Ensemble Model    | Accuracy | F1-Score |
|-------------------|----------|----------|
| LightGBM          | 0.9208   | 0.9206   |
| XGBoost           | 0.9158   | 0.9155   |
| Random Forest     | 0.9044   | 0.9042   |
| Gradient Boosting | 0.8957   | 0.8956   |
| CatBoost          | 0.9019   | 0.9018   |

---

|          |        |        |
|----------|--------|--------|
| AdaBoost | 0.2496 | 0.1713 |
|----------|--------|--------|

---

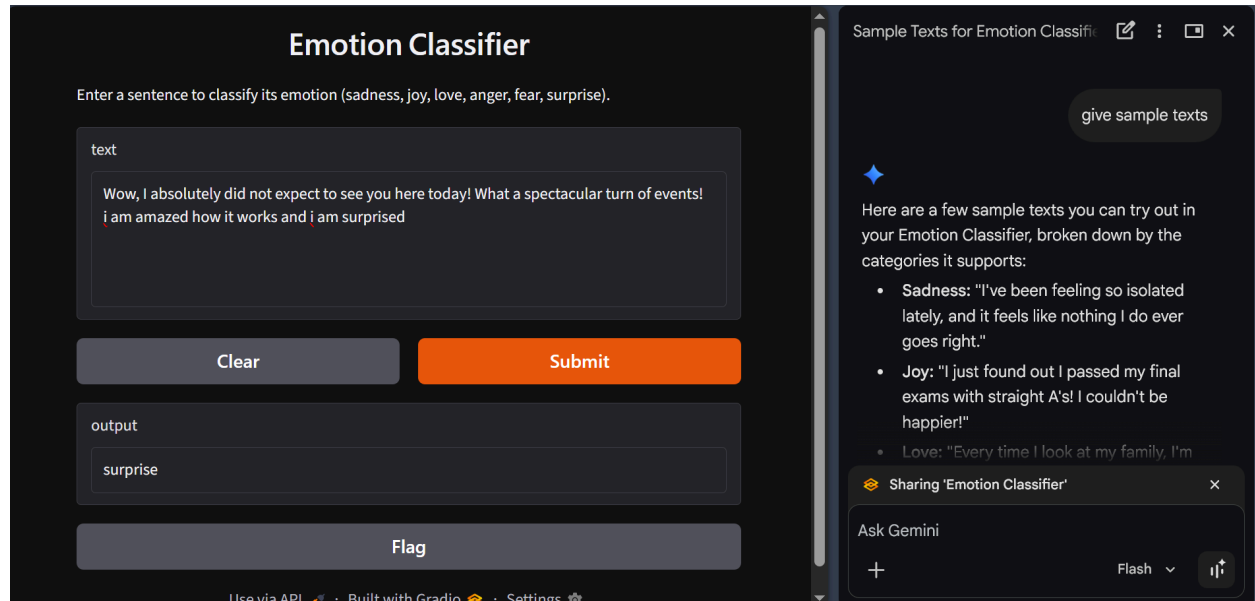
## 10. Results

### Key Findings

1. Best Overall Model: Logistic Regression with TF-IDF features achieved the highest accuracy (92.2%) and F1-score (92.1%).
  2. Feature Performance:
    - TF-IDF features consistently outperformed BoW features across all models
    - Word2Vec embeddings performed significantly worse than count-based features
    - N-grams (1,2) improved model performance by capturing contextual information
  3. Class-wise Performance:
    - "Surprise" and "Love" were the most accurately classified emotions
    - "Fear" was the most challenging emotion to classify correctly
  4. Ensemble Performance:
    - LightGBM and XGBoost performed best among ensemble methods
    - AdaBoost performed poorly, likely due to class imbalance or model complexity issues
  5. Word2Vec Limitations:
    - The Word2Vec approach performed poorly with traditional classifiers
    - This suggests that Word2Vec embeddings may require deep learning models to fully leverage their semantic representations
-

# 11. User Interface

A Gradio-based web interface was developed for real-time emotion classification:



## Interface Features

- Text Input: Users can enter any text for emotion classification
- Clear Button: Resets the input field
- Submit Button: Triggers emotion prediction
- Sample Texts: Pre-loaded examples for each emotion category

## Sample Output

| Input Text                                     | Predicted Emotion |
|------------------------------------------------|-------------------|
| "I've been feeling so isolated lately"         | Sadness           |
| "I just found out I passed with straight A's!" | Joy               |

---

|                                                           |          |
|-----------------------------------------------------------|----------|
| "Every time I look at my family, I'm happy"               | Love     |
| <hr/>                                                     |          |
| "Wow, I absolutely did not expect to see you here today!" | Surprise |
| <hr/>                                                     |          |
| <hr/>                                                     |          |

## 12. Future Improvements

The current implementation can be extended in several ways:

1. Deep Learning Integration:
    - Implement LSTM/GRU networks with pre-trained word embeddings
    - Use Transformer-based models (BERT, RoBERTa, DistilBERT)
    - Explore attention mechanisms for context-aware classification
  2. Data Augmentation:
    - Implement back-translation for generating synthetic training data
    - Use synonym replacement and random insertion techniques
    - Apply contextual word embeddings for data augmentation
  3. Feature Engineering:
    - Add sentiment lexicons and emotion lexicons as features
    - Incorporate part-of-speech features
    - Use dependency parsing for syntactic features
  4. Hyperparameter Optimization:
    - Perform extensive Grid Search or Bayesian Optimization
    - Tune regularization parameters for improved generalization
    - Optimize feature extraction parameters (max\_features, ngram\_range)
  5. Deployment:
    - Deploy as a web application using Flask or FastAPI
    - Create a mobile application for real-time emotion detection
    - Implement API endpoints for integration with other applications
  6. Multi-label Extension:
    - Extend to multi-label emotion classification
    - Handle emotions expressed simultaneously in a single text
-

## 13. Conclusion

This project successfully developed a text-based emotion classifier using traditional machine learning approaches. Multiple feature extraction techniques (BoW, TF-IDF, and Word2Vec) and classification algorithms were evaluated on a large Twitter dataset containing 416,809 labeled samples across six emotion categories.

The Logistic Regression model with TF-IDF features emerged as the best performer, achieving an accuracy of 92.2% and an F1-score of 92.1%. This demonstrates that relatively simple traditional machine learning approaches, when combined with appropriate feature engineering, can achieve excellent performance in text-based emotion classification tasks.

Key insights from this project include:

- **Feature Selection Matters:** TF-IDF with n-grams consistently outperformed other feature representations, while Word2Vec embeddings underperformed when used with traditional classifiers.
- **Model Selection:** Logistic Regression provided the best balance of performance and interpretability, outperforming more complex ensemble methods.
- **Class Imbalance:** While the dataset was relatively balanced, certain emotions like "Fear" were more challenging to classify accurately.
- **Real-world Applicability:** The Gradio-based interface demonstrates the practical utility of the emotion classifier for real-time text analysis.

The ensemble models, particularly LightGBM and XGBoost, also showed strong performance, suggesting that gradient-boosting methods could be further optimized for even better results. Future work should explore deep learning approaches with pre-trained language models like BERT, which have shown state-of-the-art performance in various NLP tasks.

Overall, this project illustrates the effectiveness of traditional machine learning approaches for emotion classification and provides a solid foundation for more advanced NLP applications.

---

## 14. References

1. TensorFlow Keras Documentation – Text Classification with LSTM.
2. Kaggle Dataset: Emotions Dataset for NLP by nelgiriye withana.
3. Scikit-learn Documentation – Feature Extraction and Classification.
4. Chollet, F. (2018). Deep Learning with Python. Manning Publications.
5. Mikolov, T., et al. (2013). Distributed Representations of Words and Phrases and their Compositionality. arXiv:1310.4546.
6. NLP with SpaCy – Text Processing Documentation.
7. Gensim Documentation – Word2Vec Model.
8. Gradio Documentation – Building Machine Learning Interfaces.
9. NLTK Documentation – Text Processing and Lemmatization.